Assignment — AI Quest: Interactive Quiz Game

An interactive, application-oriented quiz on Units I & II of Artificial Intelligence (DSC401-6)

Name	Manish Nayak
Register No.	2441442
Class	6 BSc DS
Department	Data Science & Statistics
University	Christ University, Bangalore

1. Aim

To design and develop an interactive, application-oriented quiz game covering the concepts of Units I and II of Artificial Intelligence — from the foundations of AI and intelligent agents through state-space search, heuristic search, constraint satisfaction and means-end analysis — that assesses conceptual understanding rather than memorisation, and that gives the player immediate, explained feedback on every answer.

1.1 Requirements and where they are met

#	Requirement (from the brief)	Where it is satisfied
1	Minimum 50 questions	72 questions (Appendix: separate Question Bank document)
2	All prescribed topics covered	All 20 prescribed topics; 21 tags (Hill Climbing variations included). Table 5
3	Variety of formats: MCQ, True/False, Assertion–Reason, Fill in the Blanks, Match the Following, Scenario-based, Problem-solving	All 7 formats present. Table 6
4	At least 10 real-world application/scenario questions	13 questions flagged real-world. Section 5.2
5	At least 5 search / state-space problems	11 questions flagged search. Section 5.3
6	Interactive features: timer, score, immediate feedback, hints/lifelines, difficulty levels, random questions, progress, leaderboard, final result	All nine implemented. Table 3, Section 4
7	Working quiz/game	Streamlit app (app.py). Section 3, Figures 1–10
8	Question bank with correct answers	Question Bank.docx, question_bank.json, and the in-app Question Bank page (Figure 9)
9	Brief report / screenshots on tool and game design	This report, Sections 3–4 and Section 6

Table 1. Requirements of the brief mapped to the submission

2. Introduction

Units I and II mix two very different kinds of knowledge. Unit I is largely conceptual — what makes an agent rational, how environments are classified, what the Turing Test does and does not prove. Unit II is procedural — given a graph and a heuristic, which path does a strategy actually return? A quiz that

only asks for definitions tests the first kind and misses the second. The game was therefore built around three design choices:

- **Apply, don't recall.** Most questions place a concept in a scenario (a UPI fraud detector, a Swiggy routing agent, an IRCTC CAPTCHA, exam timetabling at Christ) and ask the player to reason about it.
- **Hand-traceable search problems.** One small weighted graph with a heuristic table is reused for BFS, DFS, IDS, Greedy Best-First and A*, so the player compares how each strategy behaves on the same input — and sees greedy search return a sub-optimal path that A* avoids.
- **Every answer explained.** Each question carries a hint and an explanation; the explanation is shown immediately after answering, so a wrong answer still teaches.

3. Tool Used

The game is a **Streamlit** web application written in Python (app.py, about 570 lines), with the questions stored separately as data in question_bank.json. Streamlit was chosen over form-based tools (Google Forms, Kahoot, Quizizz) for three reasons:

- Form tools cannot render the figures the search questions need (a weighted graph, an 8-puzzle board, a hill-climbing landscape) nor implement game logic such as an adaptive difficulty or a 50:50 lifeline. Streamlit renders Graphviz graphs and charts natively.
- Keeping questions as data (JSON) separate from game logic means the same bank drives the game, the study-mode Question Bank page, the CSV export and the Word question bank.
- It deploys free to Streamlit Community Cloud from a GitHub repository, giving a public URL for the in-class demonstration (as in the sample provided).

File	Purpose
app.py	The game: setup screen, play loop, lifelines, timer, results, leaderboard, question-bank page
question_bank.json	72 questions with options, answer, accepted answers, hint, explanation, tags
build_bank.py	Source of the question bank; regenerates the JSON and checks its integrity
verify_search.py	Computes every search/state-space answer (BFS, DFS, IDS, Greedy, A*, water jug, 8-puzzle, hill climbing)
test_app.py, test_life.py	Automated tests using Streamlit's AppTest framework (Section 7)
requirements.txt	streamlit, pandas

Table 2. Project files

To run locally: pip install -r requirements.txt, then streamlit run app.py. The app opens at localhost:8501.

4. Game Design

4.1 Game flow

The game moves through three stages held in Streamlit's session state: **setup → play → results**. On the setup screen (Figure 1) the player enters a name and chooses the difficulty (Easy, Medium, Hard, Mixed or Adaptive), the number of questions (10 to all 72), exam mode (timer on/off) and the topics to include. The play stage shows one question at a time with a progress bar, live score and streak, countdown timer and lifelines (Figures 2–5). After each answer the player sees whether it was correct, the points earned with a breakdown, the correct answer and an explanation (Figures 6–7). The results stage shows the final score, accuracy, a badge, topic-wise accuracy with topics to revise, a full answer review, a CSV download and the leaderboard (Figure 8).

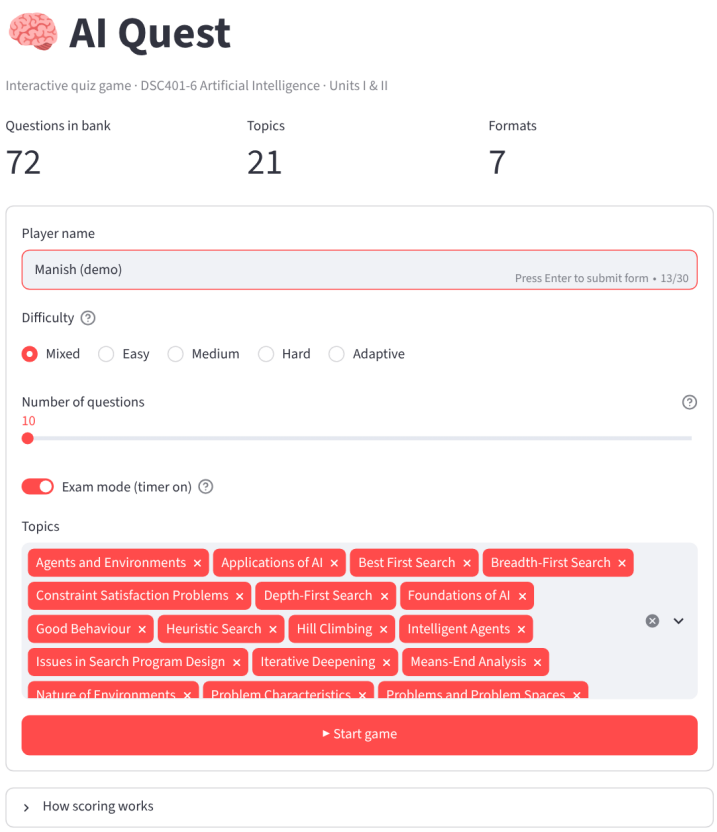


Figure 1. Setup screen: player name, difficulty, number of questions, exam mode, topic filter

4.2 Interactive features

Feature asked for	How it works in the game
Timer	Exam mode gives Easy 30 s, Medium 45 s, Hard 75 s per question (problem-solving questions are mostly Hard). A Streamlit fragment re-renders the countdown every second; at zero the answer is recorded as timed-out (wrong, 0 points).
Score calculation	Points by difficulty with lifeline penalties, speed and streak bonuses (Table 4).
Immediate feedback	Correct/incorrect banner, points breakdown, the player's answer, the correct answer and an explanation after every question.
Hints / lifelines	Hint (unlimited, halves points); 50:50 removes two wrong options (2 per game, halves points); Flip question swaps in an unseen question of the same difficulty (1 per game).
Difficulty levels	Easy / Medium / Hard / Mixed, plus Adaptive: starts at Medium, moves up a level after a correct answer and down after a wrong one.
Random questions	A fresh random draw every game, balanced across topics by round-robin sampling so a 10-question game covers 10 different topics; MCQ options are shuffled.
Progress indicator	Progress bar "Question k of n" plus live score, streak and lifelines remaining.
Leaderboard	Every finished game is appended to leaderboard.csv and ranked by score then accuracy.
Final score / result	Score, correct count, accuracy, best streak, badge, topic-wise accuracy chart, "Revise:" list and full review.

Table 3. Interactive features and their implementation

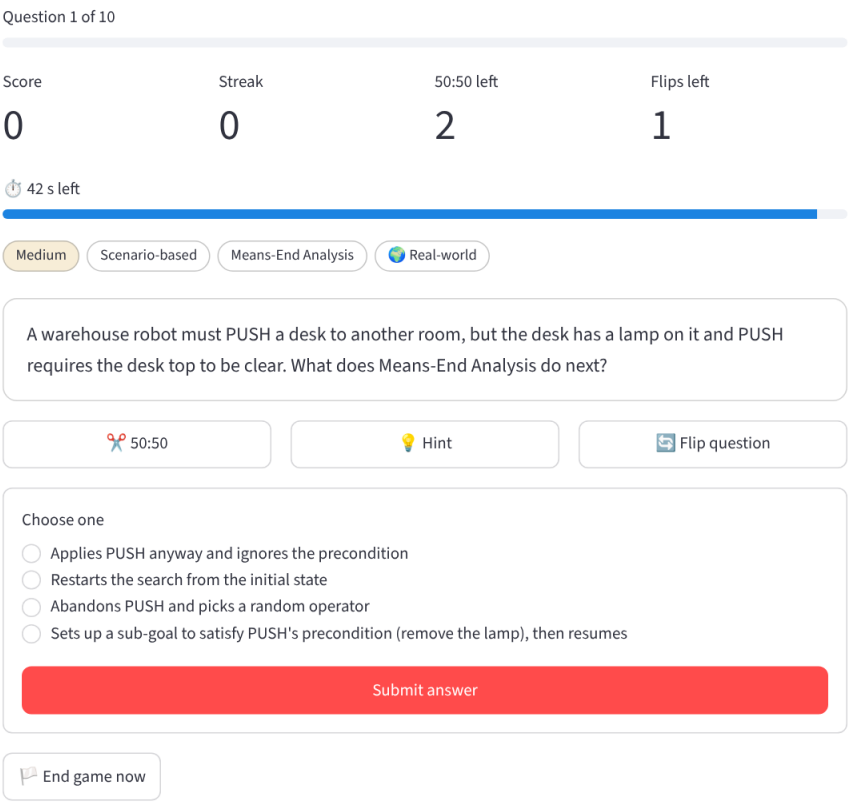


Figure 2. A question in exam mode: progress bar, score, streak, lifelines, countdown and topic/format tags

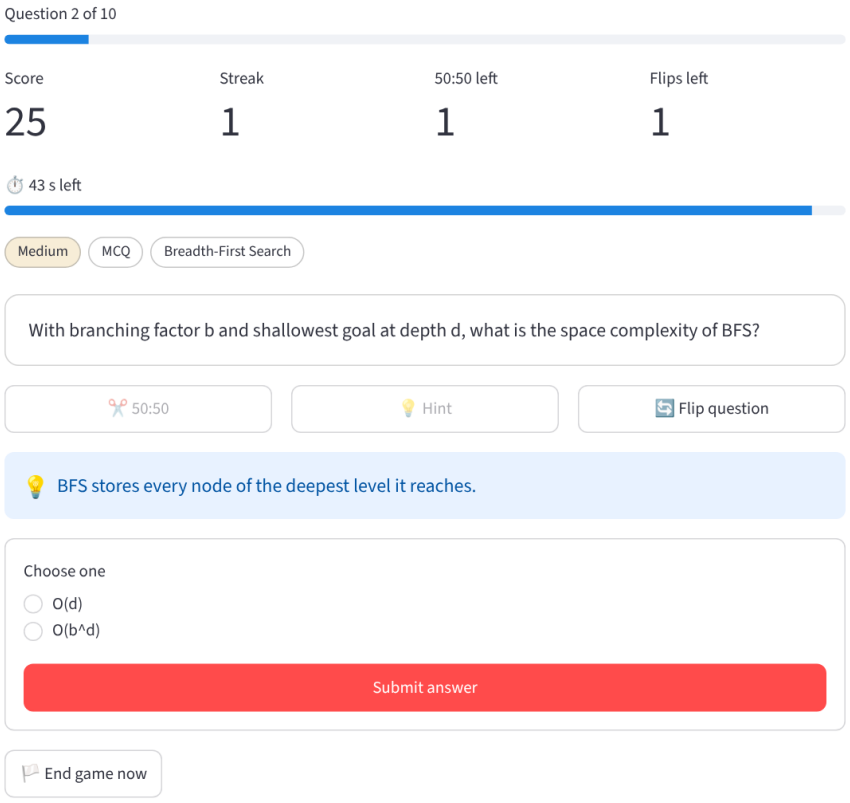


Figure 3. Lifelines in use: the hint is revealed and 50:50 has removed two wrong options

4.3 Scoring rules

Component	Rule
Base points	Easy 10, Medium 20, Hard 30
Hint used	Points for that question × 0.5
50:50 used	Points for that question × 0.5 (both lifelines: × 0.25)
Speed bonus	+5 if correct with more than half the time remaining (exam mode only)
Streak bonus	+5 on every correct answer once 3 or more are correct in a row
Wrong or timed out	0 points; streak resets
Badges (accuracy)	≥ 90% AI Grandmaster · ≥ 75% Rational Agent · ≥ 50% Heuristic Explorer · otherwise Random Walker

Table 4. Scoring rules

Weighting by difficulty rewards attempting the harder search problems, while the lifeline penalty keeps lifelines a real trade-off rather than a free answer.

4.4 Question formats in the interface

Single-answer formats (MCQ, True/False, Assertion–Reason, Scenario-based and most Problem-solving questions) use radio buttons. Fill-in-the-blank and numeric answers use a text box and are compared against a list of accepted answers after normalisation (case, spaces and hyphens ignored, so "State-Space" matches "state space"). Match-the-Following uses one drop-down per item with the right-hand options shuffled (Figure 4); it is marked correct only if every pair is right. Questions that need a figure render one: the search graph with edge costs and heuristic values (Figure 9), the 8-puzzle start and goal boards, the Australia map as a constraint graph, and the hill-climbing landscape as a bar chart (Figure 5).

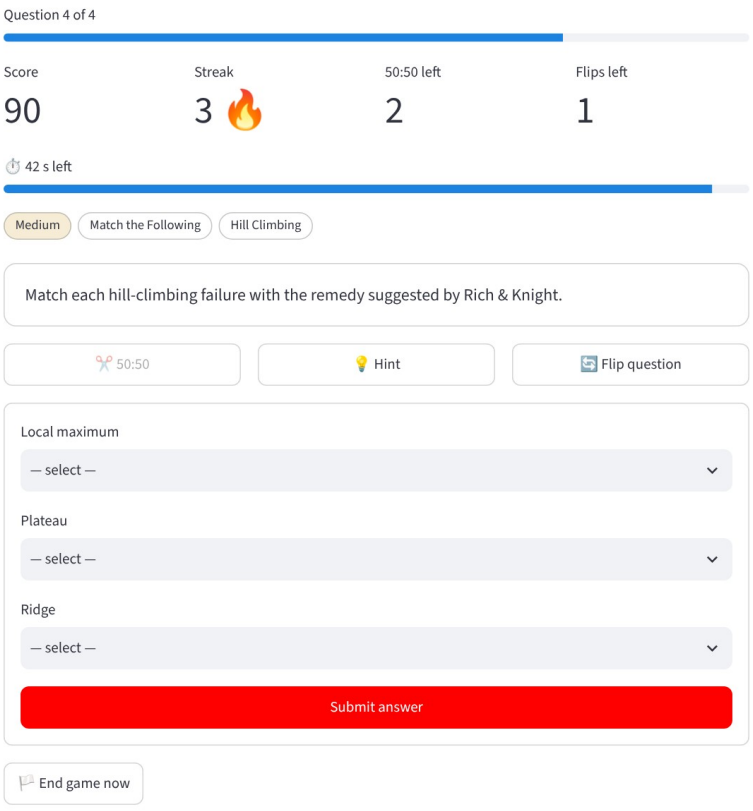


Figure 4. Match the Following rendered as one drop-down per item

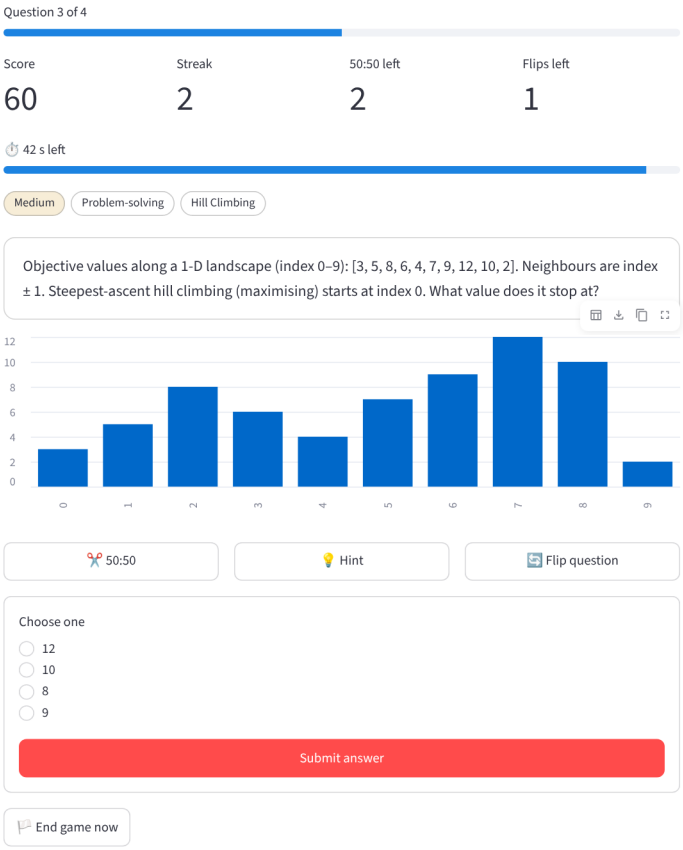


Figure 5. A problem-solving question with its figure (hill-climbing landscape)

4.5 Key code

Balanced random selection. Questions are grouped by topic, each group is shuffled, and questions are dealt round-robin across topics. A 10-question viva drill therefore touches 10 different topics instead of whatever a plain random sample happens to pick.

```
def balanced_sample(pool, n, rng):
    by_topic = {}
    for q in pool:
        by_topic.setdefault(q["topic"], []).append(q["id"])
    for ids in by_topic.values():
        rng.shuffle(ids)
    topics = list(by_topic)
    rng.shuffle(topics)
    picked = []
    while len(picked) < n and any(by_topic.values()):
        for t in topics:
            if by_topic[t] and len(picked) < n:
                picked.append(by_topic[t].pop())
    rng.shuffle(picked)
    return picked
```

Adaptive difficulty. After each answer the level moves one step up or down; the next question is drawn from that level, preferring topics seen least so far, and falls back to the nearest level if one is exhausted.

```
if s.adaptive:  # up a level after a correct answer, down after a miss
    s.level = min(s.level + 1, 2) if correct else max(s.level - 1, 0)
```

Timer. A fragment set to re-run every second redraws only the countdown, not the whole page, so the player's half-selected answer is not disturbed. When the time reaches zero it triggers one full re-run, and the main flow records the timeout.

```
@st.fragment(run_every=1)
def countdown(q):
    s = st.session_state
    if s.submitted or s.stage != "play":
        return
    left = time_left(q)
    if left <= 0:
        st.rerun()          # full rerun -> main flow records timeout
    limit = TIME_LIMIT_S[q["difficulty"]]
    st.progress(max(left, 0) / limit, text=f"{int(left)} s left")
```

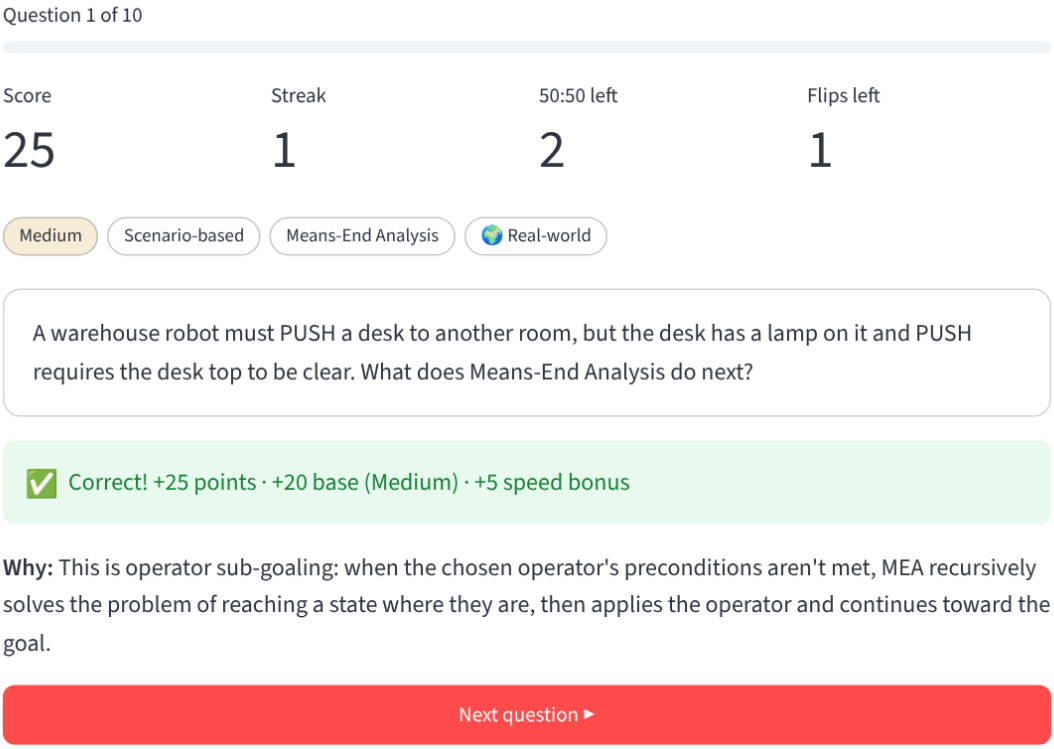


Figure 6. Immediate feedback on a correct answer with the points breakdown

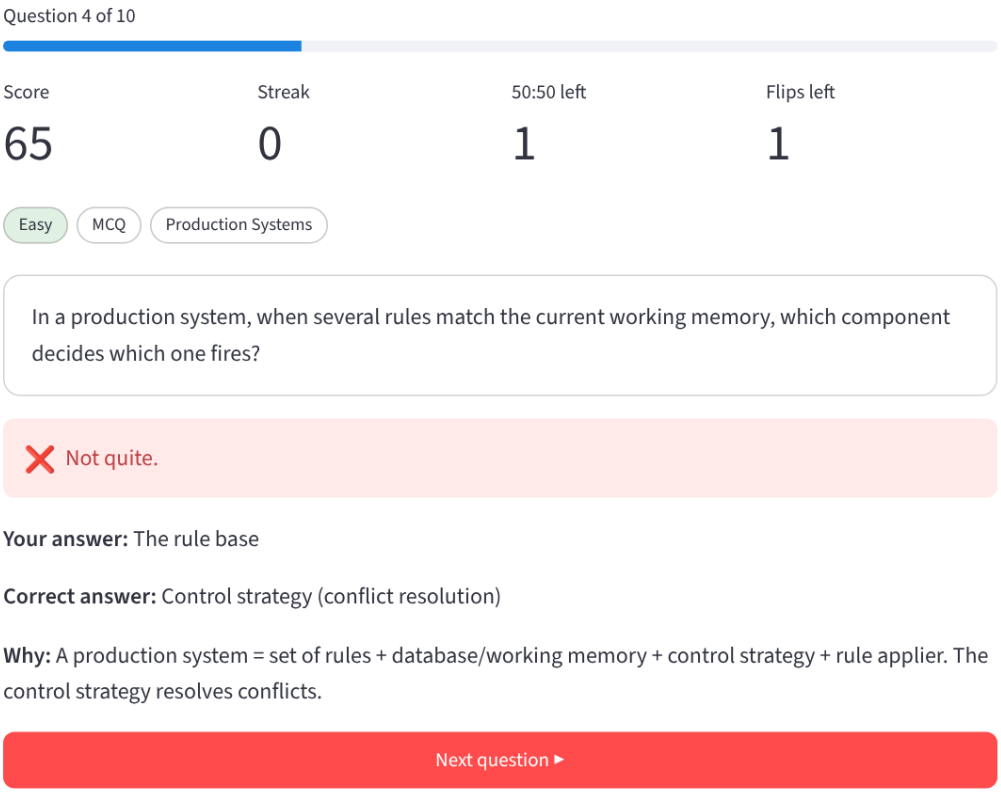


Figure 7. Feedback on a wrong answer: the player's answer, the correct answer and why

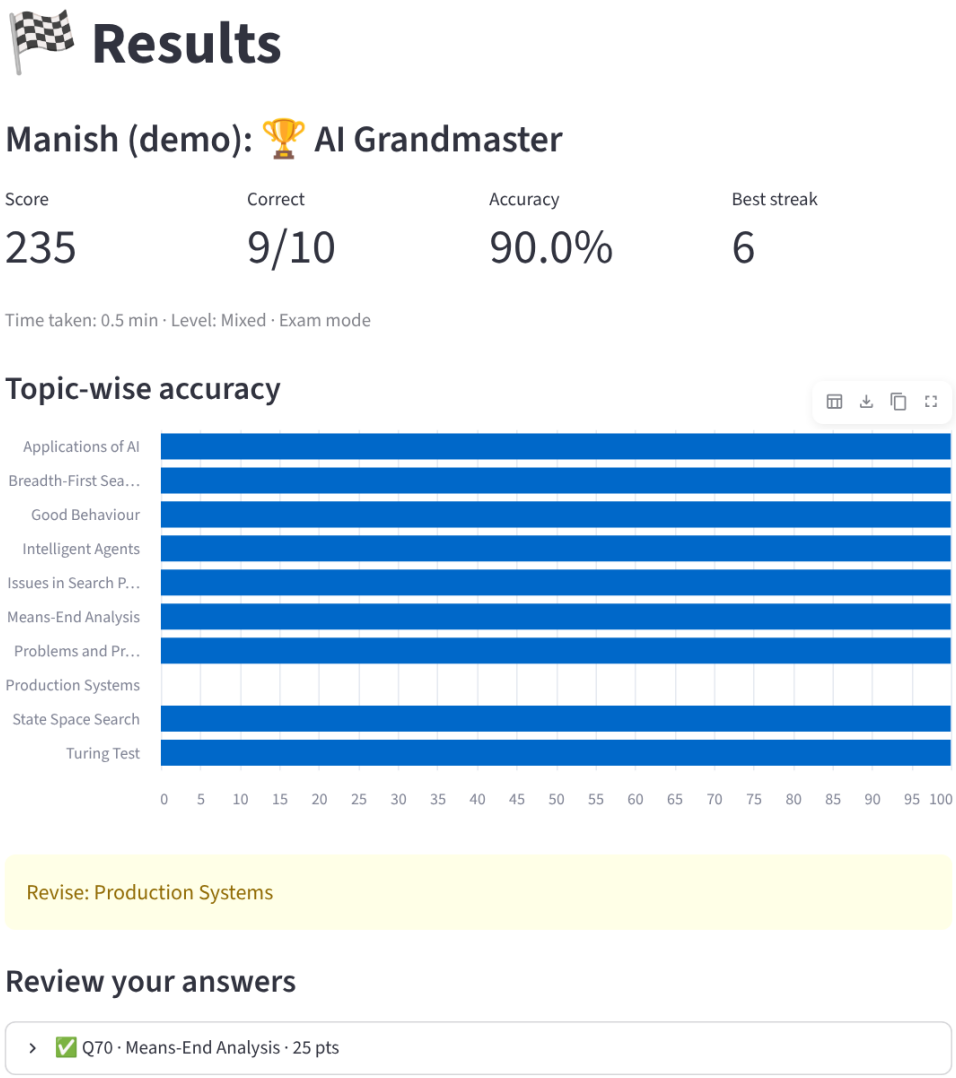


Figure 8. Results screen: score, accuracy, badge, topic-wise accuracy, topics to revise, review

5. Question Bank Design

5.1 Coverage

Topic	Qs	Question IDs
Foundations of AI	4	Q01, Q02, Q03, Q04
Applications of AI	3	Q05, Q06, Q07
Turing Test	4	Q08, Q09, Q10, Q11
Rational Agents	3	Q12, Q13, Q14
Intelligent Agents	4	Q17, Q18, Q19, Q20
Agents and Environments	3	Q21, Q22, Q23
Good Behaviour	2	Q15, Q16
Nature of Environments	4	Q24, Q25, Q26, Q27
Problems and Problem Spaces	3	Q28, Q29, Q30
State Space Search	3	Q31, Q32, Q33

Topic	Qs	Question IDs
Production Systems	3	Q34, Q35, Q36
Problem Characteristics	4	Q37, Q38, Q39, Q40
Issues in Search Program Design	2	Q41, Q42
Breadth-First Search	4	Q43, Q44, Q45, Q46
Depth-First Search	3	Q47, Q48, Q49
Iterative Deepening	3	Q50, Q51, Q52
Hill Climbing	4	Q53, Q54, Q55, Q56
Heuristic Search	3	Q57, Q58, Q59
Best First Search	4	Q60, Q61, Q62, Q63
Constraint Satisfaction Problems	5	Q64, Q65, Q66, Q67, Q68
Means-End Analysis	4	Q69, Q70, Q71, Q72

Table 5. Questions per topic

Format	Count
MCQ	15
Scenario-based	13
True/False	11
Assertion-Reason	10
Problem-solving	10
Fill in the Blank	7
Match the Following	6
Total	72

Table 6. Questions per format

Difficulty split: Easy 24, Medium 36, Hard 12. Medium dominates deliberately — Easy questions check vocabulary, while Medium and Hard ones require applying a concept or tracing an algorithm.

5.2 Real-world application questions

13 questions are set in real deployments, most in an Indian context the class will recognise: a hospital X-ray triage model, application-to-AI-field matching (Maps, Gmail, Alexa, self-driving), a UPI fraud detector evaluated on accuracy under 0.1% fraud, IRCTC CAPTCHA as a reverse Turing Test, a smart traffic signal as a simple reflex agent, PEAS for Swiggy/Zomato routing, a banking chatbot interpreting a query, LinkedIn connection degrees as BFS, iterative deepening in chess engines under a clock, random-restart hill climbing for dark-store placement, straight-line distance as an admissible heuristic in navigation, exam timetabling at Christ as a CSP, and a warehouse robot using means-end analysis.

5.3 Search and state-space problems

11 questions require working a search or state-space problem: BFS expansion order, DFS node count, IDS total expansions, the Greedy Best-First and A* paths on a shared graph, the water-jug minimum, 8-

puzzle reachable states, an invalid Missionaries-and-Cannibals state, misplaced-tile and Manhattan heuristics, a hill-climbing run and SEND + MORE = MONEY.

Verification. None of these answers was worked out by hand alone. verify_search.py implements each algorithm and prints the answer the quiz uses. While building the shared graph, the first version had a tie: S→A→C→G and S→B→D→G both cost 9, so "which path does A* return" depended on tie-breaking. The edge B→D was changed from 1 to 2 so that the optimal path is unique (cost 9; all others cost 10), and the heuristic was checked to be admissible at every node (h ≤ true cost-to-goal). Questions also state the tie-breaking rule (alphabetical) so traces are unambiguous.

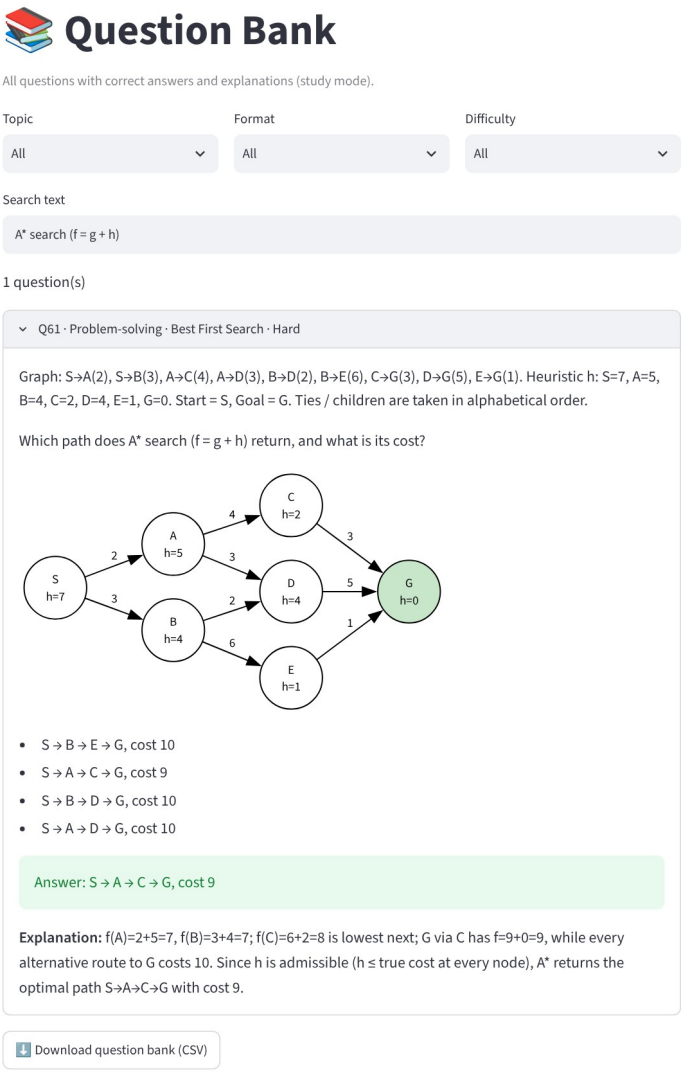


Figure 9. Study-mode Question Bank page: the shared search graph rendered with costs and heuristics, the answer and the worked explanation

6. Leaderboard and Study Mode

Besides the game, the sidebar offers a **Question Bank** page — all questions filterable by topic, format, difficulty and text, with answers and explanations, plus CSV download — and a **Leaderboard** page (Figure 10). The Question Bank page doubles as revision material for the viva.



🏆 Leaderboard

date	player	score	correct	total	accuracy	level	mode
2026-10-26 05:58	Manish (demo)	235	9	10	90	Mixed	Exam
2026-10-26 05:59	Manish (demo)	120	4	4	100	Mixed	Exam

Figure 10. Leaderboard after two demonstration games

7. Testing

The game was tested with Streamlit's AppTest framework, which runs the script headlessly and drives its widgets, and then end-to-end in a real browser with Playwright.

Test	Result
Full game over all 72 questions answering every one correctly (all 7 formats, including typed answers in upper case with trailing spaces)	All 72 marked correct; score 1,670
10 questions answered wrong in Adaptive mode	All marked wrong; level fell Medium → Easy and stayed there
10 questions answered right in Adaptive mode	Level rose Medium → Hard and stayed there
Hint + 50:50 on an Easy question answered quickly	50:50 left 2 options including the answer; 10 × 0.5 × 0.5 = 2.5, truncated to 2, + 5 speed bonus = 7 points
Timer expiry	Question recorded as timed-out, 0 points, "Time's up!" shown
Browser run (Playwright): full 10-question game with lifelines and match questions	No errors; screenshots in this report are from these runs

Table 7. Tests performed

8. Limitations

- **The leaderboard is not durable on Streamlit Cloud.** It is a CSV on the app server; Community Cloud wipes the file system when the app restarts or sleeps. It is reliable for a single demo session, not across days. A hosted database (e.g. Google Sheets or Supabase) would fix this.
- **No authentication.** Players type any name, so leaderboard entries can be duplicated or impersonated.
- **Typed answers are matched against a fixed list.** A correct synonym that is not in the accepted list (e.g. "LIFO structure" for "stack") is marked wrong. Most questions are multiple-choice partly for this reason.
- **The timer is server-driven.** The countdown refreshes over the websocket once a second; on a slow connection it can lag by about a second.
- **Adaptive mode can run short of Hard questions.** There are only 12 Hard questions; a strong player in a long adaptive game exhausts them and the game falls back to Medium.
- **Difficulty labels are the author's judgement**, not calibrated on real players. The per-question results the game records could be used to re-calibrate them.

9. Conclusion

AI Quest is a working Streamlit quiz game with 72 questions covering all prescribed topics of Units I and II in all seven required formats, with 13 real-world and 11 search/state-space questions. It implements every interactive feature listed in the brief — timer, scoring, immediate explained feedback, three lifelines, five difficulty modes including adaptive, topic-balanced randomisation, progress tracking, results analysis and a leaderboard. Every search answer was computed by code, not by hand, and the full bank was checked by an automated test that plays every question. The main open weakness is persistence: the leaderboard needs an external database to survive restarts on Streamlit Cloud.